

オペレーティングシステム論 レポート 1

情報 開発

2022311042

愛知県立大学 情報科学部 情報科学科

神野友哉

2023 年 11 月 26 日

1 はじめに

セマフォ構造体による排他制御の効果を確認する。方法として、セマフォ構造体を実装しているプログラムと、そうでないプログラムの両方に対して、処理の実施回数及び処理を適宜変更し、比較観察を行うこととする。プログラムのデフォルトの処理は、一つの共有変数 itmp の値を 1 増やす処理と、1 減らす処理をそれぞれ独立して行うスレッドを立て、itmp の最終的な値を表示する。その実行の際、PowerShell のコマンド Measure-Command にパイプ Out-Default でプログラムの出力と、その実行時間を同時に表示させる。

2 実験結果

同じ状況で、10 回計測する。両プログラムにある変数 IMAX とは、上記処理の実行回数であり、デフォルトの値 10000 でのプログラム実行結果が表 1、1000000 にしたものが表 2 である。さらに、上記処理を元のプログラムよりも短く、同様の処理が行えるように変更し、IMAX を 1000000 にして実行してみた結果が、表 3 である。表 1a と表 1b は itmp の値が 0 となっている。しかし、表 2a と表 2b、表 3a と表 3b は itmp の値が大きく異なっている。また、表 1～表 3 も処理時間の観点からみると、セマフォ構造体有りの場合、それが無い場合と比較して、長いことが見て取れる。

表 1: IMAX : 10000(デフォルト)

(a) セマフォ有り		(b) セマフォ無し	
itmp の値	処理時間 (ms)	itmp の値	処理時間 (ms)
0	69	0	30
0	67	0	8
0	72	0	8
0	77	0	7
0	70	0	8
0	78	0	8
0	79	0	8
0	81	0	8
0	102	0	9
0	88	0	9

表 2: IMAX : 1000000

(a) セマフォ有り		(b) セマフォ無し	
itmp の値	処理時間 (ms)	itmp の値	処理時間 (ms)
0	7844	-281733	34
0	7707	292051	18
0	7850	-27806	18
0	7873	247095	16
0	7931	-541874	17
0	7866	239091	17
0	5840	-249970	17
0	7804	-141585	17
0	7896	-472884	17
0	7828	318353	17

表 3: プログラム変更した IMAX : 1000000

(a) セマフォ有り		(b) セマフォ無し	
itmp の値	処理時間 (ms)	itmp の値	処理時間 (ms)
0	7959	-240390	28
0	8125	-362848	12
0	8023	310392	13
0	7900	-144753	12
0	7877	-62665	13
0	7838	411264	12
0	7818	194195	13
0	7859	11211	12
0	7880	177960	19
0	7802	663981	12

2.1 作成したプログラムについて

source 1: セマフォ有り表 1 のプログラム

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <pthread.h>

#include <semaphore.h>

#define IMAX 10000

int itmp = 0;
sem_t sema;

void countA(int x){
    int i,y;

    for ( i=0; i<IMAX; i++ ){
        sem_wait(&sema);
        y = itmp;
        y = y + 1;
        itmp = y;
        sem_post(&sema);
    }
}

void countB(int x){
    int i,y;

    for ( i=0; i<IMAX; i++ ){
        sem_wait(&sema);
        y = itmp;
        y = y - 1;
        itmp = y;
        sem_post(&sema);
    }
}

int main(void)
{
    int sinit;

    pthread_t pt1;
    pthread_t pt2;

    sinit = sem_init( &sema, 0, 1 );

    pthread_create(&pt1, NULL, (void*)countA,(void*)1);
    pthread_create(&pt2, NULL, (void*)countB,(void*)2);

    printf("in main()\n");
    pthread_join(pt1, NULL);
    pthread_join(pt2, NULL);
    printf("after pthread_join itmp: %d\n", itmp);
}
```

source 2: セマフォ無し表1のプログラム

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <pthread.h>

#define IMAX 10000

int itmp = 0;

void countA(int x){
    int i,y;

    for ( i=0; i<IMAX; i++ ){
        y = itmp;
        y = y + 1;
        itmp = y;
    }
}

void countB(int x){
    int i,y;

    for ( i=0; i<IMAX; i++ ){
        y = itmp;
        y = y - 1;
        itmp = y;
    }
}

int main(void)
{
    pthread_t pt1;
    pthread_t pt2;

    pthread_create(&pt1, NULL, (void*)countA,(void*)1);
    pthread_create(&pt2, NULL, (void*)countB,(void*)2);

    printf("in main()\n");
    pthread_join(pt1, NULL);
    pthread_join(pt2, NULL);
    printf("after pthread_join itmp: %d\n", itmp);
}
```

source 3: セマフォ有り表3のプログラム

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <pthread.h>

#include <semaphore.h>

#define IMAX 1000000

int itmp = 0;
sem_t sema;

void countA(int x){
    int i;

    for ( i=0; i<IMAX; i++ ){
        sem_wait(&sema);
        itmp++;
        sem_post(&sema);
    }
}

void countB(int x){
    int i;

    for ( i=0; i<IMAX; i++ ){
        sem_wait(&sema);
        itmp--;
        sem_post(&sema);
    }
}

int main(void)
{
    int sinit;

    pthread_t pt1;
    pthread_t pt2;

    sinit = sem_init( &sema, 0, 1 );

    pthread_create(&pt1, NULL, (void*)countA, (void*)1);
    pthread_create(&pt2, NULL, (void*)countB, (void*)2);

    printf("in main()\n");
    pthread_join(pt1, NULL);
    pthread_join(pt2, NULL);
    printf("after pthread_join itmp: %d\n", itmp);
}
```

source 4: セマフォ無し表3のプログラム

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <pthread.h>

#define IMAX 1000000

int itmp = 0;

void countA(int x){
    int i;

    for ( i=0; i<IMAX; i++ ){
        itmp++;
    }
}

void countB(int x){
    int i;

    for ( i=0; i<IMAX; i++ ){
        itmp--;
    }
}

int main(void)
{
    pthread_t pt1;
    pthread_t pt2;

    pthread_create(&pt1, NULL, (void*)countA, (void*)1);
    pthread_create(&pt2, NULL, (void*)countB, (void*)2);

    printf("in main()\n");
    pthread_join(pt1, NULL);
    pthread_join(pt2, NULL);
    printf("after pthread_join itmp: %d\n", itmp);
}
```

3 考察

IT を分かりやすく解説というブログを運営している、まつ (2022) によると、セマフォ構造体は、排他制御 (データの整合性を保つ方法) の一種であり、また、排他制御とは、共有資源に対して同時にアクセスしてもプログラムが問題なく動作できる仕組みのことだという [1]。それと 2 実験結果 を踏まえて考察する。これらのプログラムが想定している、最終的な itmp の値は 0 であるので、セマフォ無しの表 2b と表 3b は、データの整合性が保たれていないことが解かった。一方、セマフォ有りの表 1a と表 2a、表 3a は、両方とも itmp の値は 0 であった。そのため、セマフォ構造体の実装により排他制御が可能と考えられる。このレポートの 1 はじめに に記載したが、この実験の目的は、セマフォ構造体の排他制御の効果を確認することであるため、適切な方法で実験を実施できたと考えられるのではないかな。

4 感想

プログラムの処理時間を計算するコマンドから、プログラム本来の実行による標準出力を表示させるオプションを探すのに少し手間取った。セマフォ構造体有りの処理回数 IMAX を試しに 100000000 として、実行したところ、実行したまま Powershell を 30 分程度放置していても終了していなかったため、とても驚いた。

参考文献

- [1] まつ. "セマフォとは". IT を分かりやすく解説. 2022.
<https://medium-company.com/%E3%82%BB%E3%83%9E%E3%83%95%E3%82%A9/>, (参照 2023-11-226).